WOXSEN UNIVERSITY

B.Tech Artificial Intelligence and Machine Learning

Design and Analysis of Algorithms

PROJECT REPORT

Campus Shuttle Routing and Capacity Scheduling

A graph-algorithm pipeline (MST, shortest paths, greedy scheduling, dynamic programming and TSP) to reduce passenger waiting time on the Woxsen campus

Submitted by (Group project)

No.	Name	Student ID
1	D. Ritheek	25WU0102057
2	M. Srilakshmi	25WU0102140
3	B. Deekshan	25WU0102037
4	P. Priyakaran	25WU0101090
5	G. Madhan	25WU0102344

Faculty / Course instructor: [to be filled in]

Date of submission: [to be filled in]

Source code: github.com/ritheekyadav01-afk/daa-campus-shuttle

Abstract

Campus shuttles are only useful if passengers do not wait long. This project models the Woxsen University campus as a weighted graph of 14 real stops joined by 22 roads, and applies the main algorithm families of the Design and Analysis of Algorithms course to improve shuttle service: minimum spanning trees (Prim and Kruskal), single-source shortest paths (Dijkstra and Bellman-Ford), all-pairs shortest paths (Floyd-Warshall), greedy interval scheduling for driver shifts, 0-1 knapsack dynamic programming for assigning passenger groups to vehicles, and the travelling salesman problem solved exactly (Held-Karp) and approximately (nearest neighbour and an MST-based 2-approximation).

Every algorithm is implemented from scratch in Python, documented with its time and space complexity, and cross-checked against a second method. A suite of 202 automated tests passes. On the study data, the average passenger wait falls from **6.69 minutes** (one shuttle visiting stops in list order) to **2.03 minutes** (three shuttles, each looping one slice of an MST-based tour), a reduction of about **70 percent**. The MST-based tour is within **1.9 percent** of the exact optimum (10.11 against 9.92 minutes), far inside the theoretical factor of two. A fleet-size study shows that three shuttles is the point of diminishing returns.

The stop names and GPS coordinates are real. The road set, shuttle speed, passenger demand, fleet and shift data are estimates and are labelled as such in every data file and in this report; the software runs unchanged on measured data. The report presents the problem, requirements, design, algorithms with pseudocode and proofs of correctness, experiments, testing, limitations and future work.

Contents

1 Introduction
2 Problem Statement and Objectives
3 Literature Survey
4 Software Requirements Specification
5 System Design
6 Algorithms: Design, Correctness and Complexity
7 Average Wait-Time Model
8 Implementation
9 Experiments and Results
10 Testing and Verification
11 Limitations and Threats to Validity
12 Future Work
13 Conclusion
References
Appendix A Stops and Roads
Appendix B Data Files and Reproduction Steps
Appendix C Team Contributions

1 Introduction

1.1 Background

A university campus spreads over many buildings, hostels, sports grounds and gates. Students walk between them many times a day, and a shuttle service is the natural way to shorten those trips. The quality of such a service is measured by how long a passenger waits and by whether vehicles and drivers are used well. Both depend on algorithmic decisions: the order in which stops are visited, how many shuttles run, which passengers each vehicle carries, and how drivers are scheduled.

1.2 Motivation

These decisions are classic problems in algorithm design. Choosing which roads matter is a spanning-tree problem; finding the fastest route is a shortest-path problem; choosing a visiting order is the travelling salesman problem; staffing shifts is interval scheduling; and loading vehicles is a knapsack problem. The project brings these together on one realistic dataset so that each algorithm answers a concrete operational question and each result can be checked against another method.

1.3 Scope

The project covers 14 real campus stops. It builds the graph from CSV files, runs all algorithms, benchmarks them empirically, measures the effect on average passenger wait, and produces an interactive dashboard and charts. It does not model live traffic, boarding times or time-varying demand (see Section 11).

1.4 Organisation of this report

Section 2 states the problem formally. Section 3 surveys the algorithms and their sources. Section 4 gives the requirements and Section 5 the design. Section 6 describes each algorithm with pseudocode, a correctness argument and complexity. Section 7 defines the wait-time model. Sections 8 to 10 cover implementation, results and testing. Sections 11 to 13 discuss limitations, future work and conclusions.

2 Problem Statement and Objectives

2.1 Formal problem statement

Let G = (V, E, w) be an undirected weighted graph where V is the set of 14 campus stops, E is the set of 22 roads, and w(u, v) is the travel time in minutes between adjacent stops. Passenger demand is given as a set of groups, each with an origin stop, a size and a priority. A fleet of three vehicles with fixed seat capacities serves the campus, and a set of 18 driver shifts must be staffed. The task is to:

1. find the cheapest set of roads that connects all stops;
2. find the fastest route between stops, for one pair and for all pairs;
3. assign driver shifts to the smallest number of drivers;
4. assign passenger groups to vehicles so that the value carried is maximised without exceeding capacity;

5. choose a visiting order (a closed tour) for the shuttles that minimises loop time; and
6. quantify how these choices change the average passenger wait compared with a naive baseline.

2.2 Objectives

- Implement Prim, Kruskal, Dijkstra, Bellman-Ford, Floyd-Warshall, greedy activity selection, 0-1 knapsack and Held-Karp from first principles, with documented complexity.
- Verify each algorithm against an independent method or brute force.
- Reduce average passenger wait relative to a one-shuttle, list-order baseline and report the improvement honestly.
- Study how the benefit changes with the number of shuttles.
- Measure runtime empirically and compare it with the theoretical growth rate.
- Provide reproducible code, data, tests, charts and an interactive dashboard.

3 Literature Survey

This section summarises the classical results on which the project rests. All algorithms are textbook methods [1]; the survey records their origin and the property each one contributes.

3.1 Spanning trees

Kruskal [2] and Prim [3] independently described greedy algorithms that build a minimum spanning tree (MST). Both are correct because of the cut property: the cheapest edge crossing any cut of the graph belongs to some MST. Kruskal sorts edges and joins components using a disjoint-set structure; Prim grows a single tree from a start vertex using a priority queue. Both run in O(E log V).

3.2 Shortest paths

Dijkstra [4] gave the greedy algorithm for single-source shortest paths with non-negative weights; with a binary heap it runs in O((V + E) log V). Bellman [5] and Ford described the relaxation method that tolerates negative weights and detects negative cycles in O(VE). Floyd [6], building on Warshall [7], gave the O(V^3) dynamic program that computes all pairs at once.

3.3 Travelling salesman problem

The travelling salesman problem (TSP) asks for the shortest closed tour through all vertices. It is NP-hard, but Held and Karp [8] showed an exact dynamic program over subsets in O(n^2 2^n), practical for small n such as 14. When distances satisfy the triangle inequality, a preorder walk of an MST yields a tour at most twice the optimum, a result analysed by Rosenkrantz, Stearns and Lewis [9]. Nearest-neighbour is a simple greedy heuristic with no constant-factor guarantee in general.

3.4 Scheduling and allocation

The activity-selection problem (choose the largest set of non-overlapping intervals) is solved optimally by taking intervals in order of earliest finish time, proved by an exchange argument

[1]. The 0-1 knapsack problem is solved by dynamic programming in O(nW) time [1], following the principle of optimality introduced by Bellman [10]. Unlike the fractional variant, it cannot be solved by a greedy ratio rule.

3.5 Vehicle routing context

Practical transit and delivery planning is generally treated as a vehicle routing problem (VRP), a generalisation of TSP with several vehicles and capacities [11]. The present project does not solve the full VRP; it uses TSP tours split into contiguous slices as a simple, transparent multi-vehicle policy and measures its benefit. Geographic distances between stops are computed with the haversine formula [12].

3.6 Summary of methods used

Question	Method	Source	Time
Cheapest road backbone	Prim, Kruskal	[2], [3]	O(E log V)
Fastest route, one source	Dijkstra, Bellman-Ford	[4], [5]	O((V+E) log V), O(VE)
Fastest route, all pairs	Floyd-Warshall	[6], [7]	O(V^3)
Staffing shifts	Greedy earliest-finish	[1]	O(S log S)
Loading vehicles	0-1 knapsack DP	[1], [10]	O(nW)
Visiting order	Held-Karp, NN, MST preorder	[8], [9]	O(n^2 2^n), O(n^2), O(V log V)

4 Software Requirements Specification

4.1 Purpose and users

The software lets a student, instructor or campus planner load stop, road, demand, fleet and shift data, run the algorithm pipeline, and read the results as text, charts and an interactive dashboard. Primary users are the course team and the evaluators.

4.2 Functional requirements

ID	Requirement
FR1	Load stops and roads from CSV files into a weighted undirected graph, ignoring comment lines that start with #.
FR2	Compute a minimum spanning tree with both Prim and Kruskal and confirm that the totals match.
FR3	Compute single-source shortest paths with Dijkstra and Bellman-Ford and reconstruct the path to a target.
FR4	Compute all-pairs shortest paths with Floyd-Warshall and report the longest shortest trip.
FR5	Select the maximum set of non-overlapping shifts for one driver and compute the number of drivers needed to cover all shifts.
FR6	Assign passenger groups to vehicles by 0-1 knapsack, never splitting a group.
FR7	Compute closed tours by file order, nearest neighbour, MST preorder and exact Held-Karp.
FR8	Compute average passenger wait for the baseline and for the optimised service, and its sensitivity to the number of shuttles.
FR9	Benchmark runtime against graph size and plot it against the theoretical curve.

ID	Requirement
FR10	Generate an interactive HTML dashboard and PNG charts into a report folder.

4.3 Non-functional requirements

- **Correctness:** each algorithm must agree with an independent method; this is enforced by automated tests.
- **Transparency:** every function documents its time and space complexity; every assumption is stated.
- **Reproducibility:** one command (python main.py) regenerates all outputs; random tests use fixed seeds.
- **Honesty about data:** estimated inputs are marked in the data files, the README, the dashboard and this report.
- **Portability:** pure Python 3 with a small, standard dependency set; runs on macOS, Linux and Windows.

4.4 Data requirements

File	Content	Rows	Status
stops.csv	Stop id, name, latitude, longitude	14	Real names and coordinates
edges.csv	Road endpoints and travel time (min)	22	Estimated road set; time = haversine distance at 20 km/h
demand.csv	Passenger groups: origin, destination, size, priority 1-5	18	Estimated (one peak slot)
fleet.csv	Vehicle id, seat capacity, depot stop	3	Estimated (BUS-1 30, BUS-2 20, VAN-1 12)
shifts.csv	Driver shift id, start, end (HH:MM)	18	Estimated

4.5 Assumptions and constraints

- Shuttles travel at a constant 20 km/h and dwell time at stops is ignored.
- Passengers arrive uniformly in time, so the average wait at a stop served by a loop of length T minutes is T/2.
- Roads are undirected and symmetric; there are no negative or zero-length edges.
- The graph is connected, which is required by the spanning-tree and tour algorithms.

4.6 Software environment

Python 3 with matplotlib, networkx, numpy and plotly for charts and the dashboard, and pytest for testing (requirements.txt). The algorithms themselves use only the Python standard library (heapq for the priority queue).

5 System Design

5.1 Architecture

The software is organised as a pipeline of four layers. The data layer reads CSV files; the graph layer holds the adjacency-list graph and priority queue; the algorithm layer contains one module per algorithm family; and the model and reporting layer combines algorithm outputs into the wait-time model, experiments and dashboard. The driver script main.py runs the whole pipeline and prints a summary.

```
CSV data  ->  Graph (adjacency list)  ->  Algorithms  ->  Wait model &
experiments  ->  Report

data/*.csv   src/graph.py           src/mst.py       src/wait_model.py
report/dashboard.html
             (Graph, MinHeap)       src/sssp.py      src/sensitivity.py
report/*.png
                                    src/apsp.py      src/experiments.py
                                    src/greedy.py    src/dashboard.py
                                    src/dp.py
                                    src/tsp.py
```

5.2 Module responsibilities

Module	Responsibility
src/graph.py	Graph class (adjacency list), MinHeap wrapper over heapq, CSV loader that skips # comment lines.
src/mst.py	Prim (heap) and Kruskal (sorting + union-find with path compression and union by rank).
src/sssp.py	Dijkstra, Bellman-Ford with negative-cycle detection, and path reconstruction.
src/apsp.py	Floyd-Warshall with next-hop table and path reconstruction.
src/greedy.py	Earliest-finish activity selection and repeated selection to staff all shifts.
src/dp.py	0-1 knapsack table with item reconstruction and group-to-vehicle assignment.
src/tsp.py	Tour cost, nearest neighbour and Held-Karp exact solver.
src/wait_model.py	MST preorder tour, contiguous slicing for k shuttles, weighted average wait, baseline-versus-optimised comparison.
src/sensitivity.py	Average wait for k = 1 to 6 shuttles and the knee of the curve.
src/experiments.py	Random connected graphs, timing, and log-log plots against theory.
src/dashboard.py	Interactive Plotly dashboard (map, route selector, tour comparison, sensitivity).
tests/test_algorithms.py	Randomised and brute-force cross-check tests (202 test cases).

5.3 Data structures

- **Adjacency list** {u: [(v, w), ...]} gives O(V + E) space and O(deg) neighbour iteration, suitable for sparse road networks (22 roads on 14 stops).

- **Binary heap** (heapq) is the priority queue for Prim and Dijkstra, with O(log n) push and pop.
- **Disjoint sets** with path compression and union by rank give near-constant amortised find and union for Kruskal.
- **Distance matrix** from Floyd-Warshall feeds the TSP solvers and the wait model.
- **Bitmask table** dp[mask][j] is the Held-Karp state, and a two-dimensional table is the knapsack state.

5.4 Graph model

Vertices are stops S1 to S14 with real coordinates. Each edge weight is the haversine (great-circle) distance between its end stops divided by the assumed speed of 20 km/h, expressed in minutes. The edge set was chosen to link nearby stops and is an assumption, not a road survey. Figure 1 shows the graph with the minimum spanning tree highlighted.

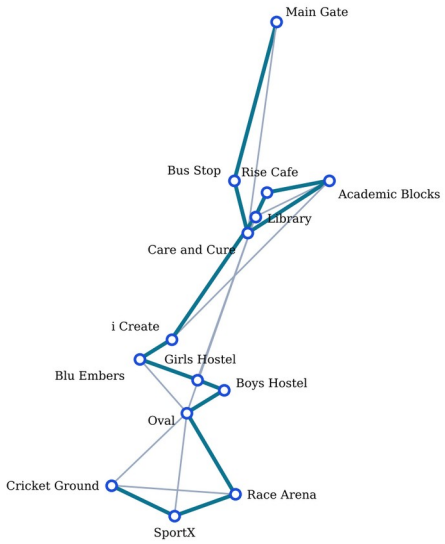


Figure 1. Campus graph on real coordinates (north at the top). Teal edges form the minimum spanning tree (total 5.81 minutes).

6 Algorithms: Design, Correctness and Complexity

6.1 Minimum spanning tree

Problem. Find a spanning tree of minimum total weight.

Kruskal

```
KRUSKAL(G):
  sort edges by weight
  for each vertex v: MAKE-SET(v)
  T = empty
  for each edge (u, v) in sorted order:
      if FIND(u) != FIND(v):
          add (u, v) to T;  UNION(u, v)
  return T
```

Prim

```
PRIM(G, s):
  visited = {s};  push all edges of s into heap
  while heap not empty and |visited| < |V|:
      (w, u, v) = pop-min
      if v in visited: continue
      add (u, v) to T;  visited += {v}
      push all edges (v, x) with x not visited
  return T
```

Correctness. Both algorithms rely on the cut property: for any cut, the minimum-weight crossing edge is in some MST. Prim always adds the cheapest edge leaving the visited set (a cut), and Kruskal always adds the cheapest edge joining two different components (also a cut), so neither ever adds an edge that is excluded from every MST.

Complexity. Prim with a binary heap is O(E log V) time and O(V + E) space. Kruskal is O(E log E) = O(E log V), dominated by sorting. **Result.** Both give 13 edges with total weight 5.81 minutes; the equality is asserted at run time.

6.2 Single-source shortest paths

Dijkstra

```
DIJKSTRA(G, s):
  dist[v] = infinity for all v;  dist[s] = 0;  push (0, s)
  while heap not empty:
      (d, u) = pop-min;  if d > dist[u]: continue
      for each (v, w) in adj[u]:
          if dist[u] + w < dist[v]:
              dist[v] = dist[u] + w;  prev[v] = u;  push (dist[v], v)
  return dist, prev
```

Bellman-Ford

```
BELLMAN-FORD(G, s):
  dist[v] = infinity;  dist[s] = 0
  repeat |V| - 1 times:
      for each edge (u, v, w): relax in both directions
  if any edge can still be relaxed: report negative cycle
  return dist
```

Correctness. Dijkstra maintains the invariant that when a vertex is removed from the heap its distance is final, which holds because all weights are non-negative. Bellman-Ford relaxes every edge |V| - 1 times, enough for any shortest path (at most |V| - 1 edges) to be fully relaxed; a further successful relaxation proves a negative cycle.

Complexity. Dijkstra: O((V + E) log V) time, O(V) space. Bellman-Ford: O(VE) time, O(V) space. **Result.** The fastest route from the Bus Stop to Race Arena is 3.3 minutes via Care and Cure, Academic Blocks, Library and Oval. Bellman-Ford returns identical distances and reports no negative cycle.

6.3 All-pairs shortest paths (Floyd-Warshall)

```
FLOYD-WARSHALL(G):
  d[i][j] = w(i, j) if edge, 0 if i = j, else infinity
  for k in vertices:
      for i in vertices:
          for j in vertices:
              if d[i][k] + d[k][j] < d[i][j]:
                  d[i][j] = d[i][k] + d[k][j];  next[i][j] = next[i][k]
  return d, next
```

Correctness. After the outer iteration for k, d[i][j] is the length of the shortest path from i to j using only the first k vertices as intermediates. This follows by induction: the best path either avoids vertex k or goes through it, giving the recurrence above.

Complexity. O(V^3) time and O(V^2) space. **Result.** The result agrees with Dijkstra, and the longest shortest trip on campus is 4.3 minutes (Cricket Ground to Main Gate). The matrix feeds the TSP solvers and the wait model.

6.4 Greedy interval scheduling for driver shifts

```
SELECT-SHIFTS(S):
  sort shifts by finish time;  chosen = [];  last_end = -infinity
  for each shift s in sorted order:
      if s.start >= last_end: chosen += [s];  last_end = s.end
  return chosen

ASSIGN-DRIVERS(S):
  while shifts remain: pick = SELECT-SHIFTS(remaining); give pick to a new
driver; remove it
```

Correctness. Let g be the earliest-finishing shift. Any optimal solution can swap its first shift for g without causing a conflict, because g finishes no later; hence a greedy first choice is safe, and the argument repeats on the remaining shifts (exchange argument [1]). For staffing all shifts, the minimum number of drivers equals the largest number of shifts that overlap at one moment; repeated selection is a greedy heuristic for this and gives 4 drivers on this data.

Complexity. SELECT-SHIFTS is O(S log S). ASSIGN-DRIVERS is O(D S log S) for D drivers. **Result.** One driver can cover 8 of the 18 shifts (SH02, SH04, SH07, SH09, SH11, SH12, SH15, SH17); 4 drivers cover all 18.

6.5 0-1 knapsack for assigning passenger groups

```
KNAPSACK(weights, values, W):
  dp[0..n][0..W] = 0
  for i in 1..n:
      for c in 0..W:
          dp[i][c] = dp[i-1][c]
          if weights[i] <= c: dp[i][c] = max(dp[i][c], dp[i-1][c -
weights[i]] + values[i])
  trace back from dp[n][W] to recover chosen items

ASSIGN-GROUPS: for vehicles in decreasing capacity: run KNAPSACK on
remaining groups
```

Model. An item is a passenger group, its weight is the group size, and its value is size x priority (priority 1 = low to 5 = high). A group is carried whole or not at all. **Correctness.** The table entry dp[i][c] is the best value using the first i groups within capacity c; the recurrence considers the two cases for group i and uses the principle of optimality. Processing vehicles one after another is a greedy decomposition of the multi-vehicle problem and is not guaranteed globally optimal, but each vehicle is loaded optimally given the groups left.

Complexity. O(nW) time and space per vehicle; O(F x G x W_max) for F vehicles and G groups. **Result.** See Table 2 in Section 9.

6.6 Travelling salesman tours

Held-Karp (exact)

```
HELD-KARP(d, s):
  dp[{j}][j] = d[s][j] for each stop j
  for each subset S in increasing size, for each j in S, for each k not in
S:
       dp[S + {k}][k] = min(dp[S + {k}][k], dp[S][j] + d[j][k])
  return min over j of dp[all][j] + d[j][s]   (and recover the order by
parent pointers)
```

Nearest neighbour and MST preorder

Nearest neighbour repeatedly moves to the closest unvisited stop (O(n^2)). The MST-based tour walks the MST in preorder from the start, visiting the nearest child first, and shortcuts repeated vertices through shortest-path distances (O(V log V) after the MST is known).

Correctness and guarantee. Held-Karp is exact because the optimal tour restricted to any subset is itself optimal for that subset. For the MST tour, a full traversal of the tree (each edge twice) costs 2 x MST; shortcutting cannot increase cost when the triangle inequality holds, which shortest-path distances satisfy; and the optimal tour minus one edge is a spanning tree, so OPT is at least MST weight. Hence the tour cost is at most 2 x OPT [9].

Complexity. Held-Karp O(n^2 2^n) time, O(n 2^n) space (feasible for n = 14, about 16,000 states per stop). **Result.** Table 3 in Section 9.

7 Average Wait-Time Model

The wait model translates algorithm outputs into the quantity passengers care about. Let a shuttle loop visit a set of stops and take T minutes for one full circuit. If passengers arrive uniformly in time, the expected wait at any stop on that loop is T / 2. With demand weights p(s) equal to the number of passengers originating at stop s, the system-wide average wait over a set of loops L is

Average wait = sum over loops l in L of [(T_l / 2) x P_l] / sum of all P_l, where P_l = sum of p(s) over stops s on loop l.

Loop time T_l is the sum of shortest-path times between consecutive stops of the loop, including the return leg. Three configurations are compared:

- **Before (baseline):** one shuttle visits stops S1 to S14 in the order of stops.csv and repeats.
- **Routing only:** one shuttle on a better closed tour (nearest neighbour, MST preorder or the Held-Karp optimum).
- **After:** the three vehicles of the fleet run concurrently. The MST preorder tour is cut into three contiguous slices of near-equal length and each shuttle loops one slice.

This policy is intentionally simple and transparent. Its effect is to shorten each loop, so each stop is visited more often.

8 Implementation

8.1 Repository layout

```
daa-campus-shuttle/
  README.md              project overview and how to run
  main.py                runs every algorithm and writes the report folder
  requirements.txt       matplotlib, networkx, numpy, plotly, pytest
  data/                  stops, edges, demand, fleet, shifts (CSV)
  src/                   one module per algorithm family (Section 5.2)
  tests/                 202 automated tests
  report/                dashboard.html and PNG charts (generated)
  docs/                  project report, final-review deck, script, viva
prep
```

8.2 Running the software

```
pip install -r requirements.txt
python main.py            # prints results, writes report/
pytest -q                 # runs the 202 tests
```

8.3 Interactive dashboard

main.py generates report/dashboard.html, a self-contained Plotly page showing the campus map on real coordinates, a selectable route (Dijkstra shortest path), the minimum spanning tree, the tour comparison, the before-and-after wait, the fleet sensitivity and the list of assumptions. It opens in any browser without a server.

8.4 Engineering notes

- Every function carries a docstring or comment stating its time (and where relevant space) complexity.
- CSV files begin with an # ESTIMATE comment line where the data are assumed; the loader skips lines starting with #, so the marker travels with the data.
- Random tests use fixed seeds (0 to 49) so that results are reproducible.

9 Experiments and Results

9.1 Graph algorithms

Quantity	Value	Cross-check
Stops / roads	14 / 22	
MST total weight	5.81 min (13 edges)	Prim equals Kruskal
Fastest route, Bus Stop to Race Arena	3.3 min via Care and Cure, Academic Blocks, Library, Oval	Dijkstra equals Bellman-Ford; no negative cycle
Longest shortest trip	4.3 min (Cricket Ground to Main Gate)	Floyd-Warshall equals Dijkstra

Table 1. Results of the graph algorithms.

9.2 Driver shifts

One driver can cover at most 8 of the 18 shifts (SH02, SH04, SH07, SH09, SH11, SH12, SH15 and SH17). Repeating the selection staffs all 18 shifts with 4 drivers.

9.3 Passenger groups and vehicles

Vehicle	Capacity	Seats used	Value	Groups carried
BUS-1	30	30	125	G06, G11, G13, G14
BUS-2	20	19	69	G01, G05
VAN-1	12	9	36	G02

Table 2. Knapsack assignment (value = passengers x priority). 58 of 127 passengers are placed in this slot; the other 11 groups wait for later trips.

9.4 Tour comparison

Tour	Loop (min)	Ratio to optimum	Avg wait (min)
File order (baseline)	13.37	1.348	6.69
Nearest neighbour	10.11	1.019	5.05
MST preorder	10.11	1.019	5.05
Optimal (Held-Karp)	9.92	1.000	4.96

Table 3. Closed-tour cost and one-shuttle average wait for each tour (start at Bus Stop).

The MST-based tour is only 1.9 percent longer than the exact optimum, well inside the theoretical bound of 2. On this data nearest neighbour and MST preorder happen to produce the same tour. The file-order baseline is 35 percent worse than the optimum, so even a one-shuttle service gains 1.6 minutes of wait from routing alone.

The optimal tour is Bus Stop, Main Gate, Care and Cure, Rise Cafe, Library, Girls Hostel, Boys Hostel, Oval, Cricket Ground, SportX, Race Arena, Blu Embers, i Create, Academic Blocks and back. The MST-based tour is Bus Stop, Care and Cure, Academic Blocks, Rise Cafe, Library, i Create, Blu Embers, Girls Hostel, Boys Hostel, Oval, Race Arena, SportX, Cricket Ground, Main Gate and back.

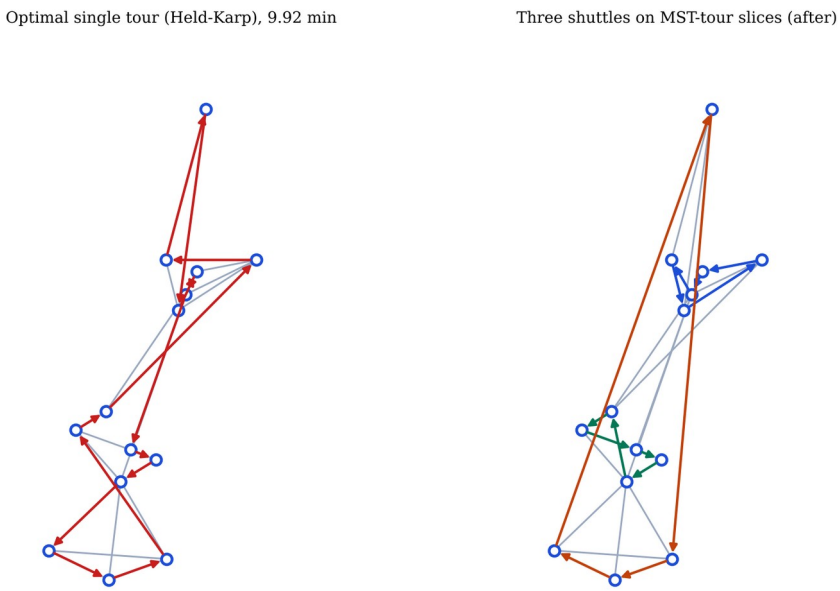


Figure 2. Left: optimal single tour (arrows show shortest-path legs). Right: three shuttles on contiguous slices of the MST tour.

9.5 Effect on average wait

Configuration	Average wait (min)	Change vs baseline
Before: one shuttle, file order	6.69	-
One shuttle, MST tour	5.05	-24 %
After: three shuttles on MST-tour slices	2.03	-70 %

Table 4. Average passenger wait for the three configurations.

The three loops are Bus Stop - Care and Cure - Academic Blocks - Rise Cafe - Library; i Create - Blu Embers - Girls Hostel - Boys Hostel - Oval; and Race Arena - SportX - Cricket Ground - Main Gate. The overall average wait falls from 6.69 to 2.03 minutes, a 70 percent reduction (69.7 percent before rounding).

9.6 Fleet-size sensitivity

Shuttles	1	2	3	4	5	6
Average wait (min)	5.05	3.91	2.03	1.75	1.26	1.12
Saving vs previous	-	1.14	1.88	0.28	0.49	0.14

Table 5. Average wait against number of shuttles (MST tour slices).

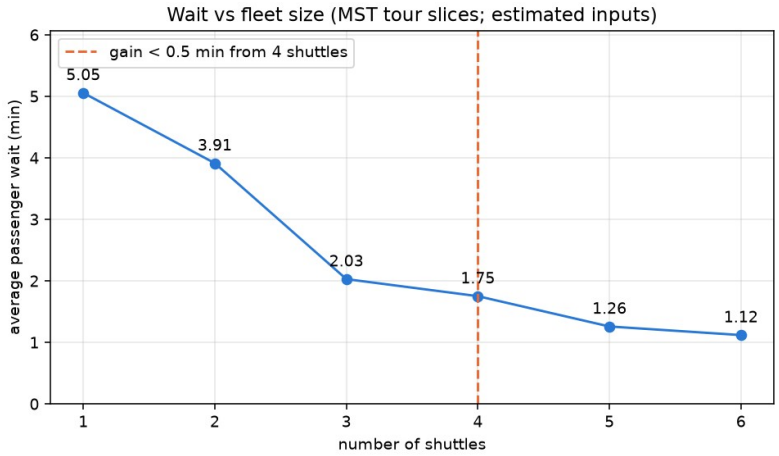


Figure 3. Average wait against fleet size. The first shuttle count whose extra vehicle saves less than 0.5 minutes is 4.

Going from two to three shuttles saves 1.88 minutes; the fourth saves only 0.28. Three shuttles is therefore the knee of the curve, and equals the size of the (estimated) fleet file. The small bump at five shuttles (0.49 saved) arises because slice lengths are not perfectly balanced; a balanced partition could smooth this.

9.7 Empirical running time

To check the complexity analysis, each graph algorithm was timed on random connected graphs of average degree 4 with 50 to 1,600 vertices (best of 3 runs; Floyd-Warshall only up to 200 vertices because of its cubic cost).

V	E	Prim (ms)	Kruskal (ms)	Dijkstra (ms)	Bellman-Ford (ms)	Floyd-W. (ms)
50	100	0.04	0.04	0.03	0.05	3.3
100	200	0.09	0.09	0.07	0.19	27.1
200	400	0.19	0.18	0.13	0.26	209
400	800	0.44	0.38	0.28	0.52	-
800	1,597	0.93	0.78	0.64	1.36	-
1,600	3,199	2.14	1.77	1.25	3.22	-

Table 6. Measured running time (milliseconds) on this development machine; absolute values vary by machine.

Doubling V roughly doubles the time of the near-linear algorithms (Prim, Kruskal, Dijkstra), as expected from E log V growth, and multiplies Floyd-Warshall time by about 8 (3.3, 27.1 and 209 ms), matching O(V^3). Figure 4 plots measured time against the scaled theoretical curve on log-log axes.

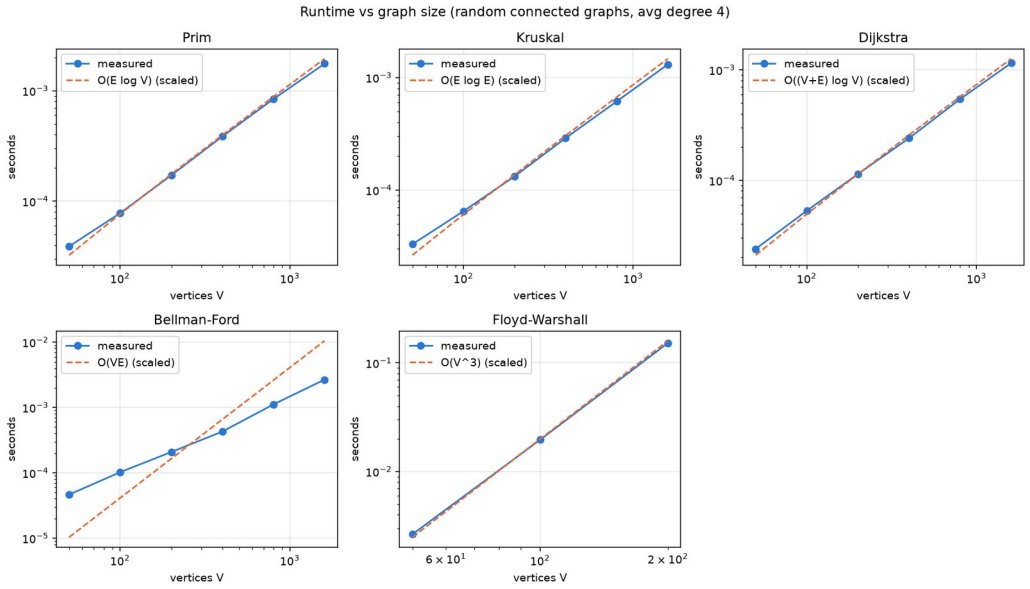


Figure 4. Measured runtime against the theoretical complexity curve (scaled by a fitted constant).

10 Testing and Verification

The test suite (tests/test_algorithms.py) contains 202 test cases and all pass. Four of the tests are parametrised over 50 random seeds (200 cases) and two are single deterministic checks.

Test	Cases	What it verifies
test_shortest_paths_agree	50	Dijkstra, Bellman-Ford and Floyd-Warshall return the same distances on random connected graphs of 5 to 25 vertices.
test_prim_equals_kruskal	50	Prim and Kruskal give equal total weight on random graphs.
test_held_karp_beats_heuristics	50	The exact tour is never longer than nearest neighbour or the MST tour, and the MST tour respects the factor-2 bound.
test_knapsack_matches_brute_force	50	The DP value equals the best subset found by enumerating all subsets.
test_held_karp_matches_brute_force	1	Held-Karp cost equals the minimum over all permutations on a small instance.
test_held_karp_on_real_campus	1	Held-Karp on the real 14-stop graph gives a valid tour no longer than any heuristic.

Independent cross-checks run on every execution of main.py as well: Prim against Kruskal, Dijkstra against Bellman-Ford, and Floyd-Warshall against Dijkstra. Beyond correctness, the report numbers in Section 9 are printed by main.py itself, so there is no manual transcription between program output and results.

11 Limitations and Threats to Validity

- **Estimated inputs.** The road set, the 20 km/h speed, the demand, the fleet and the shifts are assumptions. Absolute minutes (for example 6.69 and 2.03) depend on them; the relative gains from better routing and more shuttles are the robust findings. The software is built to accept measured data without code changes.

- **Straight-line travel times.** Edge times come from haversine distance, with no road-detour factor, traffic, speed limits or turns.
- **No dwell time or boarding model.** Stops are instantaneous; in reality each stop adds time and loop length.
- **Uniform arrival assumption.** The T/2 wait formula assumes steady, uniform arrivals; peak-hour surges are not modelled.
- **Simple multi-vehicle policy.** Slicing one tour into contiguous parts is transparent but not an optimal vehicle-routing solution; slices are balanced by stop count rather than by loop time.
- **Greedy decompositions.** Staffing all shifts by repeated selection and loading vehicles one at a time are heuristics, optimal for each step but not proven globally optimal.
- **Single scenario.** The knapsack assignment is for one peak slot, not a full timetable.

12 Future Work

- Survey real campus roads and measure real ride times, dwell times and speeds; replace the estimates in the data files and rerun.
- Collect real boarding counts by stop and time of day, and make demand time-dependent.
- Use the exact Held-Karp tour (or a balanced partition of it) as the basis for the k-shuttle policy, since it is shorter than the MST tour; balance slices by loop time instead of stop count.
- Move from slicing to a proper vehicle-routing heuristic with capacities and time windows (savings algorithm, local search).
- Add directed roads and one-way restrictions, which would exercise Bellman-Ford and Johnson's algorithm for sparse all-pairs problems.
- Provide a live visualisation of shuttle positions and a what-if tool in the dashboard.

13 Conclusion

The project demonstrates that classical algorithms from the course combine into a useful decision pipeline for a real campus. A minimum spanning tree provides a backbone and a near-optimal tour order; shortest-path algorithms give routes and the distance matrix; greedy scheduling and dynamic programming handle drivers and vehicle loading; and an exact Held-Karp solution certifies how good the heuristics are. On the study data, routing alone cuts the average wait by about a quarter, and running three shuttles on slices of the MST tour cuts it by about 70 percent, with three shuttles marking the point of diminishing returns.

Equally important are the engineering habits that make the result trustworthy: each algorithm is cross-checked against another, 202 automated tests pass, complexity is measured as well as derived, and the estimated nature of the inputs is stated wherever the numbers appear. The next step is to replace the estimates with measured campus data.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[2] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48-50, 1956.

[3] R. C. Prim, "Shortest connection networks and some generalizations," Bell System Technical Journal, vol. 36, no. 6, pp. 1389-1401, 1957.

[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269-271, 1959.

[5] R. Bellman, "On a routing problem," Quarterly of Applied Mathematics, vol. 16, no. 1, pp. 87-90, 1958.

[6] R. W. Floyd, "Algorithm 97: Shortest path," Communications of the ACM, vol. 5, no. 6, p. 345, 1962.

[7] S. Warshall, "A theorem on Boolean matrices," Journal of the ACM, vol. 9, no. 1, pp. 11-12, 1962.

[8] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," Journal of the Society for Industrial and Applied Mathematics, vol. 10, no. 1, pp. 196-210, 1962.

[9] D. J. Rosenkrantz, R. E. Stearns and P. M. Lewis II, "An analysis of several heuristics for the traveling salesman problem," SIAM Journal on Computing, vol. 6, no. 3, pp. 563-581, 1977.

[10] R. Bellman, Dynamic Programming, Princeton University Press, 1957.

[11] P. Toth and D. Vigo (eds.), Vehicle Routing: Problems, Methods, and Applications, 2nd ed., SIAM, 2014.

[12] R. W. Sinnott, "Virtues of the haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

Appendix A Stops and Roads

ID	Stop	Latitude	Longitude
S1	Main Gate	17.649906	77.799699
S2	Bus Stop	17.647008	77.798891
S3	Academic Blocks	17.647007	77.800709
S4	Library	17.646345	77.799293
S5	Rise Cafe	17.646786	77.799511
S6	Care and Cure	17.646045	77.799145
S7	Boys Hostel	17.643174	77.798692
S8	Girls Hostel	17.643364	77.798182
S9	i Create	17.644102	77.797689
S10	Blu Embers	17.643742	77.797082
S11	Oval	17.642759	77.797984
S12	Race Arena	17.641270	77.798904
S13	Cricket Ground	17.641432	77.796536
S14	SportX	17.640874	77.797746

Table A1. Campus stops (real coordinates).

Road	Time (min)	Road	Time (min)	Road	Time (min)
S1 - S2	1.00	S4 - S9	0.91	S10 - S11	0.44
S1 - S6	1.30	S4 - S11	1.27	S11 - S12	0.58
S2 - S6	0.33	S7 - S8	0.17	S11 - S13	0.64
S3 - S4	0.50	S7 - S11	0.26	S11 - S14	0.63
S3 - S5	0.39	S8 - S10	0.37	S12 - S13	0.75
S3 - S6	0.59	S9 - S10	0.23	S12 - S14	0.39
S3 - S9	1.36	S4 - S5	0.16	S13 - S14	0.43
S4 - S8	1.06				

Table A2. The 22 roads (estimated road set; time = haversine distance at 20 km/h).

Appendix B Data Files and Reproduction Steps

All inputs are in the data folder of the repository. To reproduce every number in this report:

```
git clone https://github.com/ritheekyadav01-afk/daa-campus-shuttle.git
cd daa-campus-shuttle
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python main.py     # prints all results and regenerates report/
pytest -q          # 202 passed
```

Appendix C Team Contributions

[Draft placeholder. The team should fill in the contribution of each member before submission.]

Member	Student ID	Contribution
D. Ritheek	25WU0102057	[to be filled in]
M. Srilakshmi	25WU0102140	[to be filled in]
B. Deekshan	25WU0102037	[to be filled in]
P. Priyakaran	25WU0101090	[to be filled in]
G. Madhan	25WU0102344	[to be filled in]